

Visual Patterns Associated to Abstract Trees

Uwe Kastens and Carsten Schmidt

*Universität Paderborn, Fakultät für Elektrotechnik, Informatik und Mathematik, Fürstenallee 11,
33102 Paderborn*

Abstract

Visual languages have an important role in modeling systems, specification of software, and in specific application domains. By using visual properties like spatial placement or line connections complex structures can be presented, so that humans can understand them quickly. Visual languages can be based on domain-specific metaphors, so that domain specialists can use their conventional way of description and abstraction.

For working with a visual language, a specialized graphical frontend is needed. In contrast to textual languages, general purpose editors are insufficient for visual languages, because each visual language has its particular graphical requirements. The frontend should provide methods to aid efficient drawing and restructuring of visual expressions. Often, language-specific structure editors are used as frontends for visual languages. The visual program is stored in a language-dependent data structure. The user interacts with one or more visual representations. Edit operations are directly applied to the underlying structure and after a change the graphical representation is recomputed.

The implementation of visual languages requires a wide range of conceptual and technical knowledge from issues of user interface design and graphical implementation to aspects of analysis and transformation for languages in general. We present a powerful toolset that incorporates such knowledge [16]. It generates editors from high level specifications: A language is specified by identifying certain patterns in the language structure, selecting a visual representation from a set of precoined solutions, and associating the pattern to constructs of the abstract grammar. A complete visual structure editor is generated from such a specification. It represents visual programs by attributed abstract trees. Therefore, further phases of processing visual programs can be generated by state-of-the-art tools for compiler construction. Even challenging visual languages can be implemented with reasonable small effort and with rather limited technical knowledge. The approach is suitable for a large variety of visual language styles.

Keywords: Visual languages, domain specific languages, trees, graphical user interface

1 Introduction

In this lecture we present methods and tools for the design and implementation of visual languages. The approach is based on our work in the field of language design and implementation in general for more than 20 years. One result of



Fig. 1. SIM Cards programmed in Java

that work is the *Eli* System [1,4,8]. About six years ago we started research in the field of visual languages in particular. We have taken the driving idea for our approach for visual language implementation from the *Eli* system: There computational patterns encapsulate precoinced solutions for common tasks of typing and name binding. We carried that method over to visual patterns which encapsulate visual representations of language constructs. Due to that facility our system *VL-Eli* effectively supports the design and implementation of visual languages.

Specification languages are most effective if they are designed for a specific domain. So, in Sect. 2 examples for and properties of domain specific visual languages (DSVL) are introduced. If the domain of a language is rather narrow it will be used by only a small user community. Hence, one can not afford to spend high efforts and costs for the development of a dedicated language. Effective support is needed from generators. In the third section we present some aspects on implementation strategies for domain specific languages. The methods and tools used for our approach to generate visual structure editors from visual patterns are explained in Sect. 4.

2 Domain Specific Visual Languages

We begin our presentation of DSVLs with an example taken from a project, which we made for an industrial partner, ORGA Kartensysteme GmbH. They produce chipcards which are used as so called Subscriber Identity Modules (SIMs) in mobile phones. Such a card contains a processor which is programmed in a subset of Java (see Fig. 1). The company asked us to develop a dedicated visual language for programming those cards. It should hide most of the technical programming details and should be suitable for people who do not have programming skills.

We started the language development with a thorough domain analysis: Typical SIM-Toolkit applications extend the facilities of a mobile phone by specific functions provided by the network service provider. Most of these

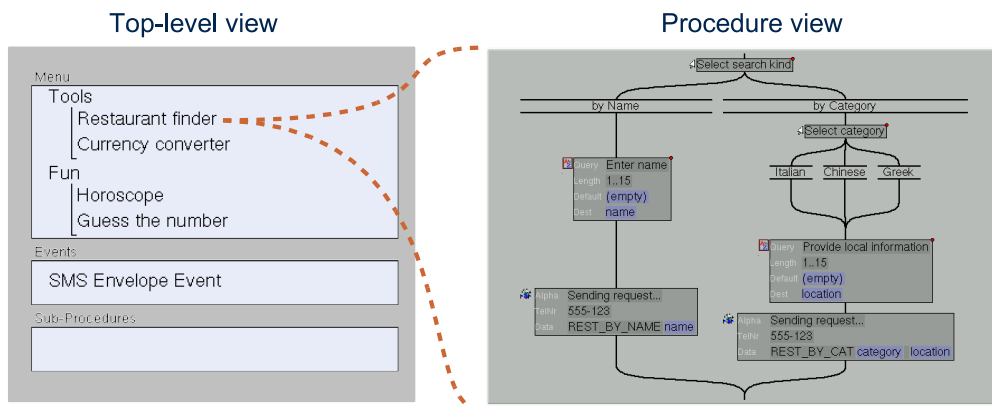


Fig. 2. Example for a visual SIM Program

functions require exchange of information between the mobile phone and the service provider via implicitly sent short messages (SMS). For example the phone user may ask a restaurant finder service for the phone number of an Italian restaurant close to his or her present location. The service provider then locates the cell phone, looks up a suitable restaurant and sends its phone number back.

Such service functions are usually organized in menus. Hence, our language provides two views (see Fig. 2): An overview of the functions in form of the menu to be implemented, together with lists of events and some auxiliary functions, and a second view presenting the control structure of a particular function.

In order to decide on the language constructs we talked to people who develop such applications and studied their notions, notations and the tools they are using. A domain specific language should be very well adapted to the way its users are used to think and develop. Such service procedures have a rather simple control-flow which mainly describes nested selections of branches. The operations on the branches, like "send an sms" or "query the phone", are easy to describe but rather complex to implement. The meaning of the visualization of the control-flow in the procedere view of Fig. 2 should be obvious.

We implemented the language by a tool consisting of a visual structure editor, an analysis phase, and a translator into the JavaCard language. All components are generated by our system. The translation had to be optimized towards the tight restrictions on storage requirements for the card processor, and it had to overcome several technical deficiencies of the target language. The domain specific language provides operations and constructs on a high level of abstraction, and it hides all the technical considerations of their im-

plementation. That makes the DSL approach so effective. Furthermore the visual appearance of the language constructs are easy to learn and to use, in particular for people who are not accustomed to program. When we demonstrated the language implementation to the developers of the company, they surprisingly uttered preference to use the specialized visual language instead of JavaCard. As a consequence they asked for additional language constructs, which would enable them to partially escape from the tight set of language elements and insert fragments of JavaCard code. We considered this move as an indication for a successful language design and implementation.

The domain specific language approach is suitable for any application domain that

- is rather narrow,
- has high-level concepts,
- needs a variety of tasks to be solved
- has notions to describe the tasks.

We developed DSLs for many domains in projects which ranged from student work up to industrial contracts. Examples are mechanical feedback systems, test descriptions for car instruments, music scores, and robot control. We consider domains that lie outside of computer science to be most challenging. For those domains the languages express notions totally different from programs and data structures. Furthermore, the potential users may not have programming skills; but the DSL approach enables them to develop software.

Typical examples for domains within computer science are generating data base reports, creation of user interfaces, and implementation of data structures from descriptions. Of course, the area of language implementation itself has examples for specialized domains that are supported by DSLs, e.g. regular expressions to specify scanners, context-free grammars to specify parsers, and dedicated languages for the toolset *Eli*. For further aspects of design and implementation of DSLs see also [7,14,2].

Textual languages usually cause their programs to be rather dense and terse. Visual notations can better illustrate the use of certain language concepts. That is particular by true for DSLs. Some domains, like Petri-Nets or Statecharts have a graphical notation which can form the basis of a visual language. There is a large variety of visual language constructs that can be used to compose a visual language. Fig. 3 shows descriptions formulated in four visual languages, which are designed in very different styles: The language *Streets* uses a metaphor of branching streets to visualize control flow. *Petri-Nets* is based on simple geometric elements, *Micro-Pascal* visualizes structured text, and the language for regular expressions combines elements of directed

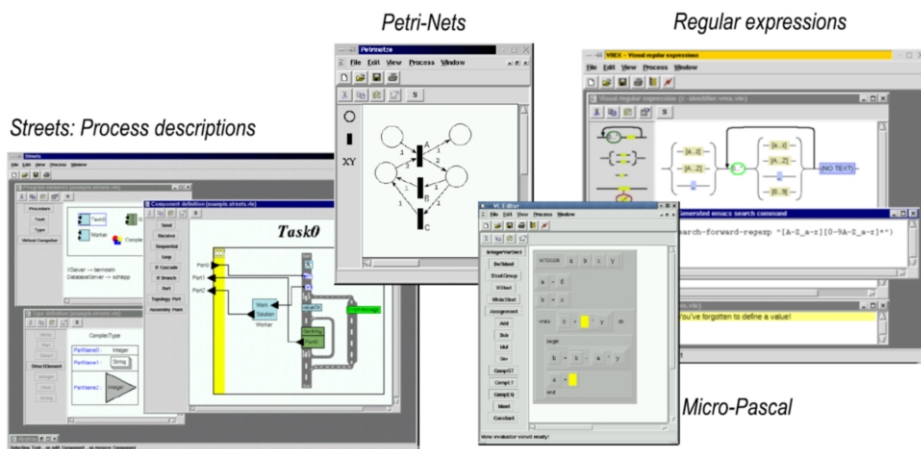


Fig. 3. Visual languages of different styles

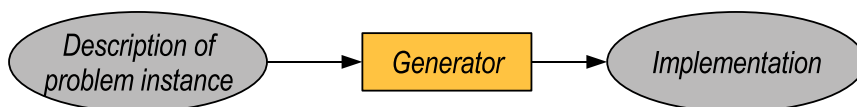


Fig. 4. Application Generator

graphs and text. With these examples we want to illustrate the expressive power of visual constructs. Of course, it is a great challenge to provide tool support, that covers such a variety of notations.

3 Language Implementation Strategies

The design and implementation of domain specific languages follows a well-known principle of software reuse. In 1988 J. C. Cleaveland [3] coined the name application generator for it (Fig. 4): Problem instances of a certain domain are described. A generator takes such a description and produces software that solves the problem. Cleaveland initially presented the approach for the domain of generating variants of reports on a data base.

The application generator approach is applicable for a certain domain

- if there are many variants of problem instances to be solved,
- they can be described without solving them, and
- the domain is completely understood, such that software solutions can be generated automatically.

The domains of generating data base reports, parsers, scanners and all the other examples of DSL domains given in the previous section fulfill these requirements. The approach is most effective if the domain has domain specific

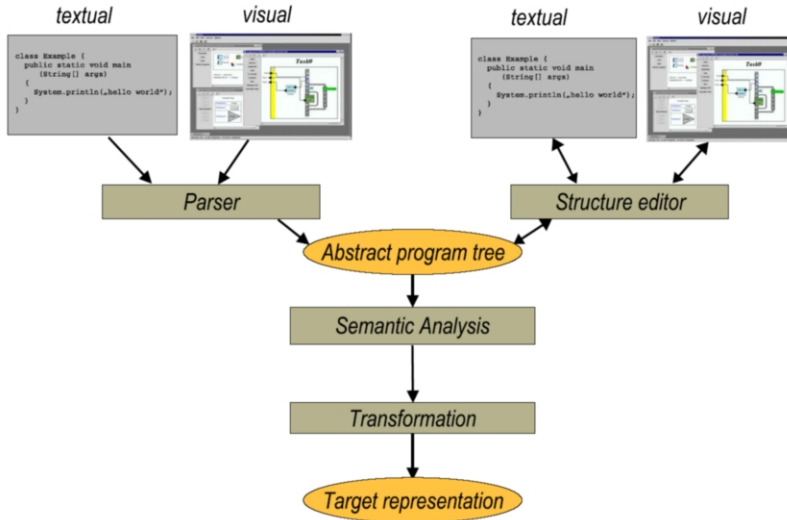


Fig. 5. Processing textual and visual descriptions

high-level notions and abstractions that can be transformed into executable software. Then the generator spans a large cognitive distance between the abstractions of the problem domain and the implementation techniques used for the solutions. Take for example a context-free grammar as a high-level description for the task of a parser, and LALR(1) as a technique applied by a parser generator. The SIM language of Sect. 1 is another example for this approach.

Such a generator encapsulates the know-how of experts, who understand both the problem domain and how to create software solutions automatically. For the users of the application generator it is sufficient to understand the notions of the problem domain. Ch. W. Kruger characterizes the generator approach as the most effective reuse method [9].

Application generator is just another name for the implementation of a domain specific language. The latter name emphasizes that a language for the problem descriptions has to be designed. Implementation of languages is a well-understood area in computer science with a body of knowledge accumulated in more than 50 years of research. Textbooks in compiler construction like [17] describe how to decompose the language implementation task, and tools like *Eli* [1,4,8] are available to solve the subtasks.

The diagram in Fig. 5 shows the structure of four variants of language implementations. They differ in two aspects:

- textual vs. visual language
- input submitted in files vs. interactive input.

The left most branch represents the structure of a conventional implementation of a textual language: A text file is analyzed by a parser. It builds an abstract program tree. Next, the phase of semantic analysis computes and checks properties of the program, e.g. binds uses to definitions, checks types, and identifies operators. The results of the semantic analysis are annotated to the tree using attributes. The final phase transforms the attributed tree into a target representation, the software solution of the described problem.

The four variants shown in Fig. 5 differ only in the way how the abstract program tree is build by the so-called front-end. The parsing approach can be applied for visual languages as well. Then a specific graphical parser processes a graphical representation. The parser derives its structure and stores it in an abstract program tree. The *DiaGen* System of M. Minas [10,11,12] supports this parsing approach. However, it represents the program as a graph, rather than as a tree.

The two right branches of Fig. 5 represent front-ends that support interactive creation of the input. A structure editor offers operations to insert, delete, or modify substructures of the program constructed so far. The editor is specialized for the particular language. That means, it writes or draws the fixed parts of language constructs, and it guides the user to compose programs that have a correct structure. Such structure editors exist for textual languages [15] as well as for visual languages.

In any of the four cases the front-end builds an abstract tree. It is the central data structure for the subsequent phases. Hence, the phases of semantic analysis and transformation can be implemented using standard methods and tools for visual languages as well as for textual ones.

In the rest of this paper we present methods and tools for the implementation of visual languages using dedicated, interactive structure editors as front-ends, i.e. the right-most branch in Fig. 5. They are generated by our *VL-Eli* system. It is build on top of the *Eli* system. The latter is used to generate the phases of semantic analysis and transformation. Fig. 6 show the layers of the *VL-Eli* toolset: On the basic layer *Eli* provides general tools for language processing, *Tcl/Tk* [13] is used to implement the graphical user interface of the structure editors, and the constraint solver *Parcon* [5] contributes implementation techniques for certain strategies of graphical layout.

The VL-Generator in the middle layer accepts descriptions of visual language constructs and implements a structure editors for them using the tools on the level below.

The topmost level contains a library of visual patterns. Each of them represents a generic concept of visual language constructs. It can be instantiated to create a concrete visual construct. Such a pattern encapsulates all the tech-

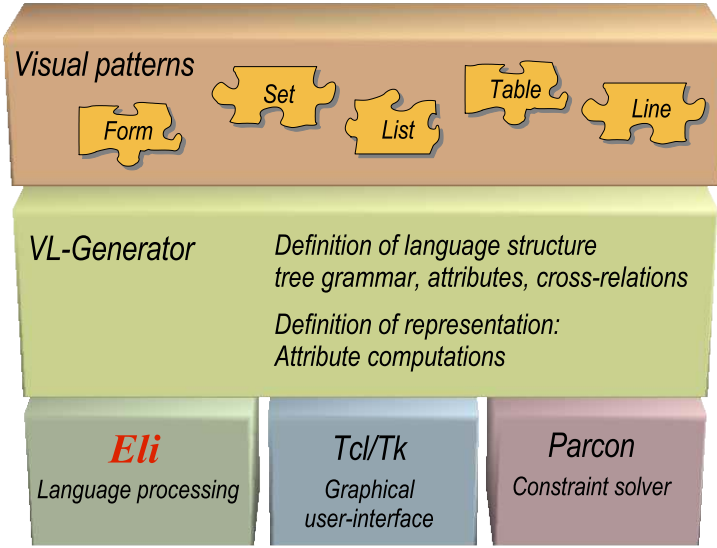


Fig. 6. The VL-Eli toolset

nical knowledge that is necessary to specify and to implement such a language construct using the levels below. This approach is described more deeply in the next section.

4 Generating Visual Editors: Visual Patterns associated to Abstract Trees

Visual patterns are the central concept of our approach for designing visual languages and generating structure editors for them. The language developer identifies certain patterns for each construct of his visual language, instantiates a precoinced solution from a library and associates it to the part of the abstract syntax which represents the construct. The structure editor is generated such that it executes the instantiated operations on the abstract program tree to visualize the construct and to interact with them.

An abstract program tree is used as a central data structure for representation of the input of compilers and translators. Those trees are specified by the abstract syntax, i.e. a context-free grammar which can be systematically derived from the concrete syntax, in case of textual languages. In a language processor a parser or a structure editor determines the structure of the input and represents it by an abstract program tree. Subsequent phases of the translator walk through the tree and decorate it with attributes. They represent properties of the program constructs they are attached to, e.g. the type of an expression. Some attributes relate a program construct to a description of an

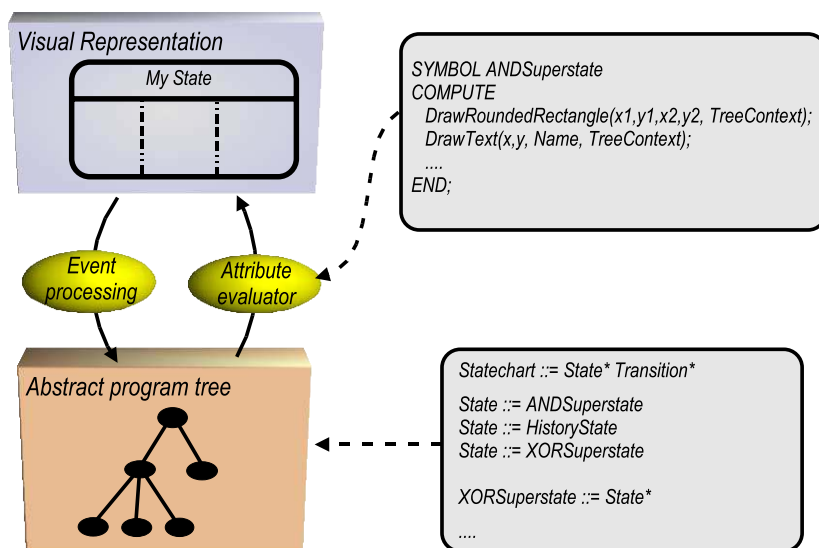


Fig. 7. Computations in tree contexts

entity defined in the program, e.g. a variable or a function. Other attributes may relate several program constructs to one another.

All these techniques are well-accepted as standards for the implementation of textual languages. Most of them can serve their purpose for visual languages, as well: The structure of a visual program can be represented by a tree, that is specified by an abstract syntax. Attributes of tree nodes represent drawings of program constructs, their changeable parameters, and their locations on the 2-dimensional drawing pane. The layout of visual program constructs is achieved by computations, which are associated to tree contexts and determine the location attributes. The approach is suitable for different strategies of layout computations: from formatting by a simple tree walk to connecting a constraint solver.

Visual languages often use graphical means, like lines or connectors, to relate program constructs, where the same relation is expressed by name binding in textual languages. Such relations usually do not obey the underlying tree structure. However, they can be implemented for visual languages, too, by attributes that refer to tree nodes or to descriptions of entities, as in the textual case.

This approach is illustrated in Fig. 7. It visualizes an abstract program tree for a statechart. The tree structure is specified by an abstract syntax for the visual statechart language. A subset of its production is shown in the lower right diagram. The diagram on the upper right specifies two computations which are associated to the grammar symbol **ANDSuperstate**. They draw the bounding rectangle and print the name of the state. These computations are

executed by an attribute evaluator when it visits on its tree walk a node of kind **ANDSuperstate**. Executing computations on a tree-walk as specified for all kinds of tree nodes individually is a general principle of attribute evaluation. It is applied in the structure editor for

- drawing the graphical elements,
- computing the layout properties,
- processing user interactions,
- computing and checking properties of language constructs, and
- transforming them into a target text.

Systems like Liga [6] generate such tree-walking attribute evaluators from attribute grammar specifications, which associate attributes and computations to contexts of the abstract program tree. The *VL-Eli* system is based on this method and is specialized to generate visual structure editors.

On top of *VL-Eli* we developed a layer of reusable visual patterns. For that purpose we analyzed a large number of visual languages of various styles. We could classify all their language constructs in a small number (about 10) visual concepts: For example the visual concept **List** is characterized by drawing an ordered sequence of elements side by side such that the whole list forms a rectangle. The layout is influenced by the graphical requirements of the elements and those of the context where the whole list is embedded in. Interactive operations are provided to insert an element into the list and to delete one from it. Properties like the orientation of the sequence, west to east, north to south, etc. are left open to be decided on instantiation, as well as the graphical representation of the separator between elements.

In our system a visual pattern is a generic specification that encapsulates a visual concept, parameterized computations to produce the graphical representation, to layout its elements, and to perform the interaction operations. **Set**, **Form**, **Line**, and **Table** are further examples for visual patterns. Their specifications are collected in a library to be used by the designer and implementor of a visual language (Fig. 8).

We consider the **ANDSuperstate** of statecharts to demonstrate how visual patterns are used. Any **ANDSuperstate** is to be represented by a rectangle as shown in the upper left part of Fig. 9. That is a form which consists of two fields, one for the name the other for the list of regions. That requirement matches to the **Form** pattern of the library. We associate an instance of it to the symbol **ANDSuperstate** in the abstract syntax. According to its production an **ANDSuperstate** consists of an **ASName** and an **ASRegionList**. They play the role of **FormElements** for this instance of the **Form** pattern. The instance is specialized by the computation of a certain graphic for the representation

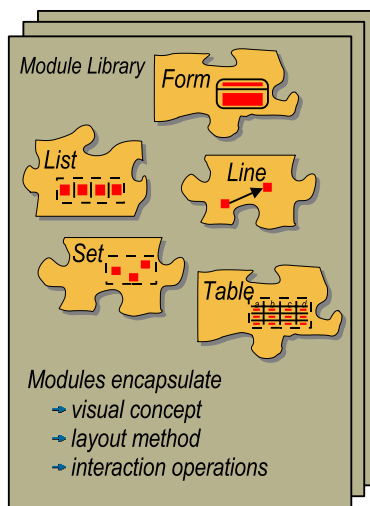


Fig. 8. Library of visual patterns

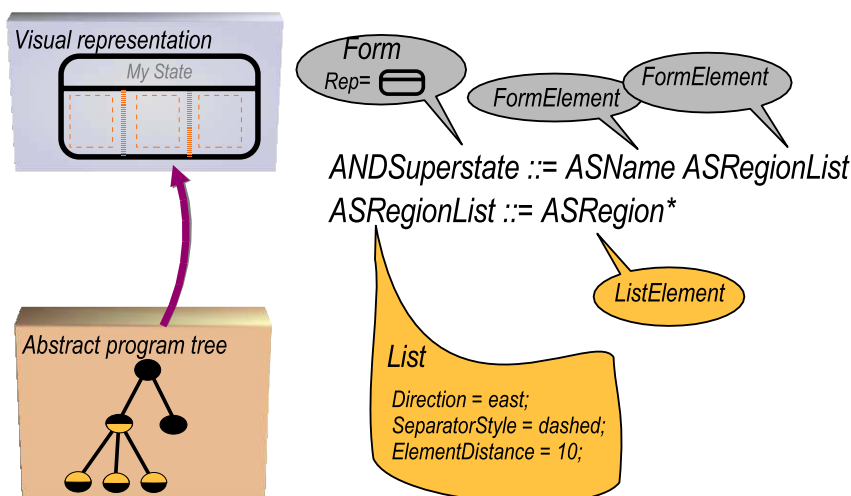


Fig. 9. Using visual patterns

attribute `Rep`. Similarly, the `List` pattern is instantiated for `ASRegionList` with `ASRegion` in the role of `ListElements`. In this case the instance is specialized by its direction and parameters which determine how elements are separated.

The generic parameters allow to create a large variety of different instances from a visual pattern. They influence the graphical representation as shown in the example of Fig. 9. One can also determine more complex graphical properties, e.g. select one out of three different layout strategies. We generated structure editors for several visual languages of very different styles using our library of visual patterns. It turned out that for each of them the whole set

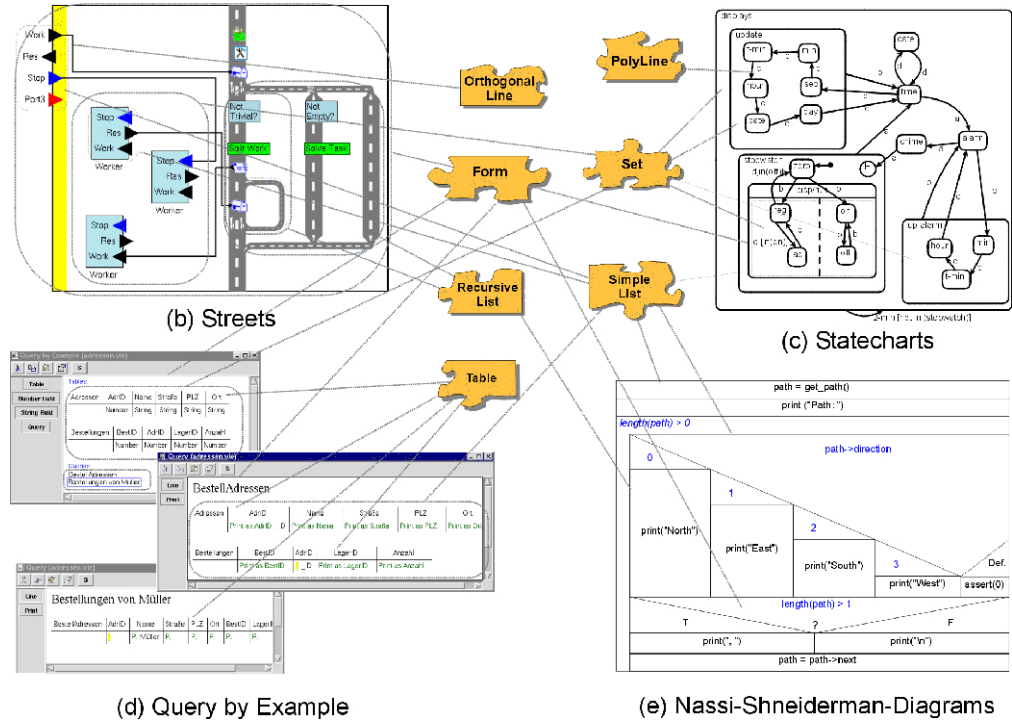


Fig. 10. Occurrences of patterns in different languages

of language constructs could be completely covered by instances of patterns from the library. Fig. 10 indicates occurrences of patterns in examples of four of those languages.

If the design of a visual language and the implementation of a structure editor for it can be achieved by just instantiating a visual pattern for each language construct, then a huge amount of implementation effort is avoided by reusing code from libraries. Especially implementations of the following tasks need not be done manually

- management of graphical data structures,
- drawing and redrawing,
- layout algorithms,
- interactive insertion and deletion,
- user interface.

Even more important, the language implementor need not understand the techniques to solve these tasks. The effort of the language implementor can be characterized by

- develop the abstract syntax,

- select and instantiate a visual pattern for each language construct,
- parametrize the instantiations, and
- contribute elementary graphical elements.

For the languages shown in Fig. 3 these contributions amount to the following sizes of specifications:

- Streets: 2460 lines
- Petri-Nets: 220 lines
- Micro-Pascal: 570 lines
- Regular expressions: 540 lines

The sizes are roughly proportional to the number of productions in the abstract syntax. They do not include the implementation of translator phases after the structure editor.

5 Conclusion

In this lecture we presented an approach for the design of visual languages and their implementation. The central concept is the selection of visual patterns and their instantiation for language construct. The visual patterns provide reusable specifications and implementations of tasks to be solved by visual structure editors. The patterns are associated to the constructs of the abstract syntax of the visual language. The generated structure editors build abstract program trees to represent the program. Hence, standard techniques of language processing can be applied to solve further translation tasks beyond editing.

The approach is supported by a layered system of tools. It is based on approved tools for language processing like *Eli*, and graphical user interfaces (*Tcl/Tk*). The toolset has been used successfully in several realistic projects. We plan to make it available to the public as soon as a sufficient amount of user manuals has been developed. The approach and the toolset has proven to be useful for numerous visual languages of different graphical styles. In particular, it fits very well to the design and implementation of domain specific languages.

References

- [1] *Eli Website*. http://www.uni-paderborn.de/fachbereich/AG/agkastens/eli_home.html.
- [2] *Proc. 1st ACM-SIGPLAN Workshop on Domain-Specific-Languages, DSL '97*, Paris, France, jan 1997.

- [3] J. C. Cleaveland. Building application generators. *IEEE Software*, 5(4):25–33, July 1988.
- [4] R. W. Gray, V. P. Heuring, S. P. Levi, A. M. Sloane, and W. M. Waite. Eli: A complete, flexible compiler construction system. *Communications of the ACM*, 35(2):121–131, Feb. 1992.
- [5] P. Griebel et al. Integrating a constraint solver into a real-time animation environment. In *1996 IEEE Symposium on Visual Languages*, pages 12–19. IEEE Comp. Soc. Press, 1996.
- [6] U. Kastens. An attribute grammar system in a compiler construction environment. In *Proceedings of the International Summer School on Attribute Grammars, Application and Systems*, volume 545 of *Lecture Notes in Computer Science*, pages 380–400. Springer Verlag, 1991.
- [7] U. Kastens and P. Pfahler. Compositional design and implementation of domain-specific languages. In R. N. Horspool, editor, *IFIP TC2 WG 2.4 Working Conference on System Implementation 2000: Languages, Methods and Tools*, pages 152–165. Chapman & Hall, 1998.
- [8] U. Kastens, P. Pfahler, and M. Jung. The Eli system. In *Proceedings 7th International Conference on Compiler Construction CC'98*, number 1383 in *Lecture Notes in Computer Science*, pages 294–297. Springer Verlag, Mar. 1998.
- [9] C. W. Krueger. Software reuse. *ACM Computing Surveys*, 24(2):131–183, June 1992.
- [10] O. Köth and M. Minas. Generating diagram editors providing free-hand editing as well as syntax-directed editing. In *Joint APPLIGRAPH/GETGRATS Workshop on Graph Transformation Systems (GraTra'2000)*, 2000.
- [11] M. Minas. *Spezifikation und Generierung graphischer Diagrammeditoren*. Shaker-Verlag, 2001.
- [12] M. Minas. Concepts and realization of a diagram editor generator based on hypergraph transformation. *Science of Computer Programming*, 44(2):157–180, Aug. 2002.
- [13] J. K. Ousterhout. Tcl: An embedable command language. In *Proceedings of the USENIX Association Winter Conference*, 1990.
- [14] P. Pfahler and U. Kastens. Language design and implementation by selection. In *Proc. 1st ACM-SIGPLAN Workshop on Domain-Specific-Languages, DSL '97, Paris, France, January 18, 1997*, pages 97–108. Technical Report, University of Illinois at Urbana-Champaign, 1997.
- [15] T. W. Reps and T. Teitelbaum. *The Synthesizer Generator: A System for Constructing Language Based Editors*. Springer-Verlag, 1989.
- [16] C. Schmidt and U. Kastens. Implementation of visual languages using pattern-based specifications. *Software - Practice and Experience*, 35(2):121–131, Nov. 2003.
- [17] W. M. Waite and G. Goos. *Compiler Construction*. Springer, New York, 1984.